

EpiPen

Major project report submitted to CUSAT in partial fulfillment of the requirements for the
award of degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

Submitted by,

ADITHYA A R (20222506)

EEVA JOSEPH (20222531)

HARIGOVIND M G (20222533)

MARIYA K J (20222539)

Under the guidance of

Dr Preetha Mathew K.

Head of Department, Division of CSE

Miss. Anukartha V R

Associate Professor, Division of CSE

Mrs. Alice Joseph

Associate Professor, Division of CSE



**Division of Computer Science & Engineering
COCHIN UNIVERSITY COLLEGE OF ENGINEERING KUTTANAD
ALAPPUZHA
APRIL 2025**

EpiPen

Mini project report submitted to CUSAT in partial fulfillment of the requirements for the
award of degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

Submitted by,

ADITHYA A R (20222506)

EEVA JOSEPH (20222531)

HARIGOVIND M G (20222533)

MARIYA K J (20222539)

Under the guidance of

Dr Preetha Mathew K
Head of Department, Division of CSE

Miss Anukartha V R
Associate Professor, Division of CSE

Mrs Alice Joseph
Associate Professor, Division of CSE



Division of Computer Science & Engineering COCHIN
UNIVERSITY COLLEGE OF ENGINEERING KUTTANAD
ALAPPUZHA
APRIL 2025

Division of Computer Science & Engineering
COCHIN UNIVERSITY COLLEGE OF ENGINEERING
KUTTANAD, ALAPPUZHA



BONAFIDE CERTIFICATE

This is to certify that the project report entitled "EPIPEN-Food Allergy Detection via Ingredient Scanning App" has been submitted by Harigovind M G, Eeva Joseph, Adithya A R, Mariya K J in partial fulfilment of the requirements for the award of the Degree B.Tech in COMPUTER SCIENCE AND ENGINEERING of COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY.

.....
Miss. Anukartha V R
Project Coordinator

.....
Mrs. Alice Joseph
Project Coordinator

.....
Dr. Preetha Mathew K
Head Of Department Division of CSE

ACKNOWLEDGEMENT

We take this opportunity to sincerely thank all those who supported and guided us throughout the successful completion of our final year project, “EPIPEN – Food Allergy Detection via Ingredient Scanning App.”

First and foremost, we express our heartfelt gratitude to Dr. Asaletha R, Principal of Cochin University College of Engineering Kuttanad, for providing us with all necessary facilities and a conducive environment to carry out this project.

We are deeply thankful to Dr. Preetha Mathew K, Head of the Department of Computer Science, for her consistent encouragement and administrative support.

We extend our special thanks to our Project Coordinators, Mrs. Alice Joseph and Miss. Anukartha V.R, Assistant Professors, Department of Computer Science, for their valuable guidance, constructive suggestions, and unwavering support throughout the course of this project. Their inputs played a crucial role in shaping the direction and quality of our work.

We also wish to acknowledge the faculty and staff members of the Department of Computer Science for their timely help and insightful feedback at various stages of development.

Finally, we would like to thank all our peers and well-wishers who provided moral support and inspiration, enabling us to complete this endeavor successfully.

Abstract

Food allergies are a significant public health concern, posing risks of severe reactions from even trace amounts of allergens. To address this, our project “EPIPEN - Food Allergy Detection via Ingredient Scanning App” offers a mobile application that enables users to scan ingredient lists using their smartphone camera. Using Optical Character Recognition (OCR) and machine learning models trained on allergen databases, the app detects common allergens, including those masked under scientific terms. It also features personalized allergen profiles, allergy medication management (like EpiPens), and emergency guidance. Developed with React Native and Expo, EPIPEN provides real-time allergen detection and comprehensive allergy management, significantly enhancing user safety and confidence.

TABLE OF CONTENTS	Page No
1. Chapter 1: Introduction	1
1.1 Background	
1.2 Problem Statement	
1.3 Objectives	
1.4 Scope of the Project	
1.5 Significance and Advantages	
2. Chapter 2: Literature Review	3
3. Chapter 3: System Requirements	7
2.1 System Requirements	
3.2.1 Hardware Requirements	
3.2.2 Software Requirements	
2.2 Functional Requirements	
2.3 Non-functional Requirements	
3. Chapter 4: System Design	8
3.1 Use Case Diagram	
3.2 Activity Diagram	
3.3 Sequence Diagram	
4. Chapter 5: Backend Implementation	13
4.1 Directory Structure	
4.2 main.py	
4.3 routes.py	
4.4 deps.py	
4.5 API Endpoints	
allergens.py	
auth.py	
ocr.py	
scan_history.py	
4.6 Routes Assembly	
4.7 Core Files	
database.py	
security.py	

4.8 Services	
allergen_detector.py	
4.9 Model Training Process	
Data Collection & Preprocessing	
Model Choice & Architecture	
Evaluation & Metrics	
4.10 Environment & Setup	
4.11 Conclusion of Implementation	
5. Chapter 6: Frontend Overview	40
5.1 Technology Stack & Framework (React Native, Expo)	
5.2 Screen Flows	
Auth Screens (Sign Up / Login)	
Home Screen	
Scan Screen	
Results / Allergen Detection	
Manual Entry Screen	
Medicine Tracker	
Emergency Info Screen	
5.3 Navigation & Layout	
5.4 Styling & Components (Optional)	
5.5 User Experience Flow	
5.6 Conclusion of Frontend	
6. Chapter 7: Screenshots	44
6.1: Login Page	
6.2: Home Page	
6.3: OCR Scan Product Label Page	
6.4: Scan Results Page	
6.5: User Profile Page	
6.6: Medicine List Page	
6.7: Emergency Steps Page	
7. Chapter 8: Conclusion And Future Work	53
8. Chapter 9: References	54

TABLE OF FIGURE	Page No
4.1 Use Case Diagram	8
4.2 Activity Diagram	9
4.3 Sequence Diagram	11
7.1 Login Page	44
7.2 Home Page	45
7.3 OCR Scan Product Label Page	46
7.4 Scan Result Page	47
7.5 User Profile Page	48
7.6 Medicine List	49
7.7 Emergency Steps Page	50

Chapter 1: Introduction

1.1 Background

Food allergies pose a significant health challenge worldwide, affecting millions of individuals across various age groups. Even a trace amount of an allergen—such as peanuts or dairy proteins—can lead to severe, sometimes life-threatening reactions. Traditionally, individuals rely on **manual label checks** to avoid problematic ingredients. However, these manual checks can be **time-consuming** and **prone to human error**, especially when dealing with unfamiliar or ambiguous ingredient names. As the variety of packaged food products expands and global supply chains introduce foreign-language labels, it becomes increasingly difficult for consumers to confidently identify allergens. Modern technology, especially in the realm of computer vision (Optical Character Recognition, OCR) and machine learning, provides an avenue to **automate** and **streamline** allergen detection from product labels.

1.2 Problem Statement

Existing solutions often only address one or two facets of allergen detection (e.g., scanning via OCR, or basic allergen lookup) but rarely provide a **holistic** approach. Users may struggle with:

1. **Hidden Allergen Terminology:** Terms like “casein” (related to milk) or “albumin” (related to eggs) are not always recognized by simple keyword matching.
2. **Manual Checks:** Relying on personal knowledge leads to errors, particularly under time pressure or when dealing with foreign labels.
3. **Absence of Comprehensive Management:** Many apps do not integrate key features like medicine tracking (e.g., EpiPen), emergency plans, or user-specific allergen profiles.

Hence, there is a need for a comprehensive system that combines label scanning, advanced text parsing (via machine learning), user-specific allergen tracking, and additional management features.

1.3 Objectives

The primary objectives of this project are to:

1. Real-Time Allergen Detection

- Use **OCR** to extract text from product labels, then detect allergens quickly and accurately.
- Provide a manual ingredient entry option for scenarios where scanning is unavailable.

2. Machine Learning for Hidden Allergen Terms

- Go beyond simple keyword matching by employing a **multi-label classification** model that recognizes synonyms or scientific names of allergens (e.g., “casein” for milk).

3. Medicine Management and Emergency Plan

- Track allergy medications (e.g., auto-injectors, antihistamines) and store their dosage/expiration dates.
- Offer an **emergency to-do list** or contact info if an allergic reaction is suspected.

4. User-Friendly and Inclusive

- Provide a fluid mobile interface with minimal friction—suitable for day-to-day use.
- Accommodate user-specific allergen profiles, ensuring personalization without excessive overhead.

1.4 Scope of the Project

● Inclusions

- **OCR Scanning:** Implementation of a robust pipeline (using Tesseract or similar) to recognize text from images.
- **Allergen Detection:** Multi-label classification to detect multiple allergens in a single product label.
- **Manual Entry:** Text-based input for non-scannable scenarios.
- **User Authentication:** Basic sign-up/login flows to maintain user-specific profiles (allergens, medicine info).
- **Medicine Management:** A rudimentary tracker for allergy-related medications, optionally including reminders.
- **Emergency Plan:** Outlines key steps in an allergic reaction scenario and stores emergency contacts.

1.5 Significance and Advantages

1. **Increased Safety:** Real-time detection mitigates the risk of missing critical allergen information on labels.
2. **Time Efficiency:** Rapid OCR scanning reduces the need for lengthy manual checks.
3. **Personalization:** Maintains each user’s allergy profile, ensuring accurate, customized warnings.
4. **Holistic Management:** Integrates medicine tracking and an emergency plan in one app—avoiding the need for multiple disconnected tools.

Chapter 2: Literature Review

Food allergies pose a growing public health challenge, affecting an estimated 200–250 million people worldwide. As food formulations become more complex and supply chains stretch across multiple countries, accurately identifying allergenic ingredients has grown exponentially more difficult (Smith and Kumar [1]). In many cases, a single ingredient list can contain dozens of additives, preservatives, and flavorings—any one of which might harbor a hidden allergen. Moreover, labeling regulations vary by region: peanut derivatives, for example, may be listed under terms such as “arachis oil,” “hydrolyzed vegetable protein,” or “lecithin” in different jurisdictions. For consumers coping with life-threatening allergies, these inconsistencies present a very real risk (World Allergy Organization [2]).

To address these challenges, an emerging body of research focuses on automated solutions that layer **Optical Character Recognition (OCR)** for rapid, on-the-fly text extraction with **machine learning** for intelligent allergen classification. Such systems promise to transform everyday routines—whether scanning a breakfast cereal box in the grocery aisle, confirming ingredients on a restaurant menu, or even verifying product details while shopping online—into safe and accessible experiences for allergic individuals.

The Growing Need for Automated Allergen Detection

Over the past twenty years, the prevalence of food allergies has approximately doubled, among both children and adults, with anaphylaxis rates rising in parallel (World Allergy Organization [2]). Traditional allergen avoidance strategies—manually reading labels, memorizing lists of “safe” ingredients, or contacting manufacturers—are time consuming, error prone, and impractical for visually impaired users. Even highly educated consumers can miss critical warnings: the term “caseinate” may not intuitively register as a milk allergen, and “albumin” might obscure its connection to eggs (Nguyen and Rossi [3]).

Against this backdrop, automated detection systems offer a compelling alternative. By harnessing the smartphone cameras that nearly all consumers carry, real-time OCR can capture ingredient text instantly. Coupled with a robust backend that applies natural language processing and classification models, these applications can highlight potential allergens in seconds, guiding at-risk individuals away from dangerous products and reducing the burden on hospital and emergency services.

Optical Character Recognition (OCR) in Allergen Detection

At its core, OCR technology transforms an image of text—whether a high-resolution scan or a smartphone snapshot—into machine-readable characters. While OCR has long been deployed in domains such as archival document digitization and automated license-plate recognition (Lee, Park, and Kim [4]), food-label scanning introduces unique complications:

1. **Lighting Variations:** A grocery store aisle with mixed fluorescent and natural lighting can produce glare or uneven shadows, causing characters to blur or disappear from the captured image. Modern mobile-camera sensors mitigate this to some extent, but advanced preprocessing remains essential (Nguyen and Rossi [3]).
2. **Image Quality:** Shaky hands, moving crowds, or low-end smartphone lenses can create motion blur or low-contrast images that obscure small print. Research shows that a

simple Gaussian blur filter can actually improve recognition by smoothing noise while preserving text edges (Kwon and Lee [5]).

3. **Complex Layouts & Fonts:** Ingredient lists often pack dozens of items into narrow columns with tiny fonts. Products may employ stylized typography or wrap text around curved containers like cans or bottles, confounding linear OCR systems (Amrutha and Rani [6]).
4. **Multilingual Labels:** In global markets, a single product label may include English, Spanish, French, and more—sometimes within the same ingredient line. Multilingual OCR engines trained on a variety of scripts ensure broad coverage but require careful language detection and segmentation (Tanaka and Hayashi [7]).
5. **Dynamic Packaging:** Seasonal or promotional packaging can radically change a label’s background, introducing patterns or graphics that overlap text. Traditional threshold-based methods often misclassify such backgrounds as characters, necessitating advanced filtering.

Advanced OCR Techniques

Researchers have tackled these hurdles with an array of enhancements:

- **Image Preprocessing:** Converting to grayscale reduces color distractions, while histogram equalization and Gaussian blur filters help isolate text from background noise (Smith and Kumar [1]).
- **Adaptive Thresholding:** Otsu’s method automatically selects an optimal binary threshold per frame, segmenting text more accurately across varying lighting conditions (Otsu [8]).
- **Morphological Operations:** Applying dilation and erosion in sequence removes stray speckles and reconnects thin strokes, sharpening the appearance of characters (Sourvanos et al. [9]).
- **Deep Learning Models:** Convolutional Neural Networks (CNNs), and more recently Vision Transformers (ViTs), can be trained end-to-end on labeled packaging datasets. These models learn to recognize text regardless of font style or layout, significantly improving recall on decorative or curved labels (Tanaka and Hayashi [7]).

Machine Learning for Allergen Detection

Once ingredient text is extracted, the next challenge is **semantically** identifying allergens—even when they are not named exactly. Early systems relied on simple keyword-matching dictionaries, flagging terms like “peanut” or “soy” when present. However, a single allergenic protein may appear under dozens of alternate names: “arachis,” “hydrolyzed vegetable protein,” and “lecithin” all point to peanuts but would escape a basic matcher (Manning, Raghavan, and Schütze [10]).

Modern Classification Techniques

1. **Rule-Based Systems:** Use regular expressions and context rules (e.g., “contains” vs.

“may contain traces”) to improve precision. While more flexible, they demand continual updates as product formulations and labeling conventions evolve (Nguyen and Rossi [3]).

2. **Supervised Learning:** Random Forest classifiers trained on tens of thousands of annotated ingredient lines learn to detect allergen patterns even within complex phrases. Studies report >95% F1-scores for multi-label classification when sufficient training data is available (Breiman [4]).

3. **Neural Networks:** Recurrent Neural Networks (RNNs) and Transformer-based architectures (e.g., BERT) capture the sequential context of ingredient lists. They discern between hard statements (“contains peanuts”) and advisory phrases (“may contain traces”), reducing false alarms (Artificial Intelligence applications in allergic rhinitis diagnosis).

4. **Ensemble & Transfer Learning:** Combining multiple models—Random Forest, Support Vector Machines (SVMs), and Gradient Boosting—reduces both false positives and false negatives. Transfer learning from large language models (e.g., BERT, RoBERTa) further improves performance when allergen-specific training data is limited (Tanaka and Hayashi [7]).

Comparative Analysis of Existing Systems

Privacy and Security in Allergen Detection

Handling user-specific health data—such as personalized allergen profiles and scan history—demands strict privacy safeguards:

End-to-End Encryption: All data in transit and at rest must be encrypted following standards like AES-256 (GDPR).

User Consent & Control: Apps should allow users to export or delete their data at any time, in compliance with GDPR and HIPAA.

Secure Cloud Processing: When OCR or ML inference runs in the cloud, secure enclaves and strict access controls prevent unauthorized data access.

Regular Audits: Third-party security reviews ensure vulnerabilities are quickly identified and patched.

Case Studies

1. **SafeBite AR Prototype:** An academic demonstration that overlays allergen icons on live camera view; includes direct dial links to emergency contacts.

2. **McMahon et al. (2013):** Showed that AR-based training improved allergen recognition in students with intellectual disabilities, with retention at six weeks post-training.

3. **Shaer et al. (2022):** Explored AR visualizations of genomic allergen markers, enhancing patient understanding of genetic risk factors.

Future Trends in Allergen Detection

Emerging technologies promise ever more seamless, personalized, and private solutions:

AR Wearables: Smart glasses that annotate allergens in real time, hands-free and in the user's natural field of vision.

Edge AI: On-device neural networks (e.g., TinyML) deliver sub-second inference with zero cloud dependency, maximizing privacy.

Voice-Activated Scanning: Voice assistants initiate scans and read out allergen alerts to visually impaired users.

Personalized Allergen Feeds: Integration with electronic health records to auto-update profiles based on the latest medical diagnoses and lab results.

Conclusion

This chapter has traced the rapid evolution of OCR and machine learning techniques for allergen detection, from simple keyword matchers to advanced deep-learning ensembles. Comparative analyses highlight trade-offs between speed, accuracy, and maintenance. Crucially, emerging edge-AI and AR technologies will soon enable truly seamless, real-time allergen alerts while stringent privacy and security measures safeguard user trust. Our project integrates these best practices into a high-accuracy, personalized app designed for diverse real-world scenarios—transforming allergen detection from a manual chore into an accessible, reliable service.

Chapter 3: System Requirements

Requirement Analysis

3.1 System Requirements

3.1.1 Hardware Requirements for the User

3.1.1.1 **Smartphone** (Android or iOS) with a working camera.

3.1.1.2 **Minimum RAM**: 2GB (recommended) for smooth OCR and app performance.

3.1.1.3 **Internet Connectivity**: Required for accessing the server.

3.1.2 Software Requirements

3.1.2.1 **Mobile OS**: Android or iOS.

3.1.2.2 **Application Dependencies**:

3.1.2.2.1 React Native environment.

3.1.2.2.2 Tesseract.js for OCR.

3.1.2.2.3 OpenCV libraries for image preprocessing.

3.1.2.2.4 FastAPI backend.

3.1.2.2.5 PostgreSQL database engine.

3.1.3 Hardware Requirements During Development

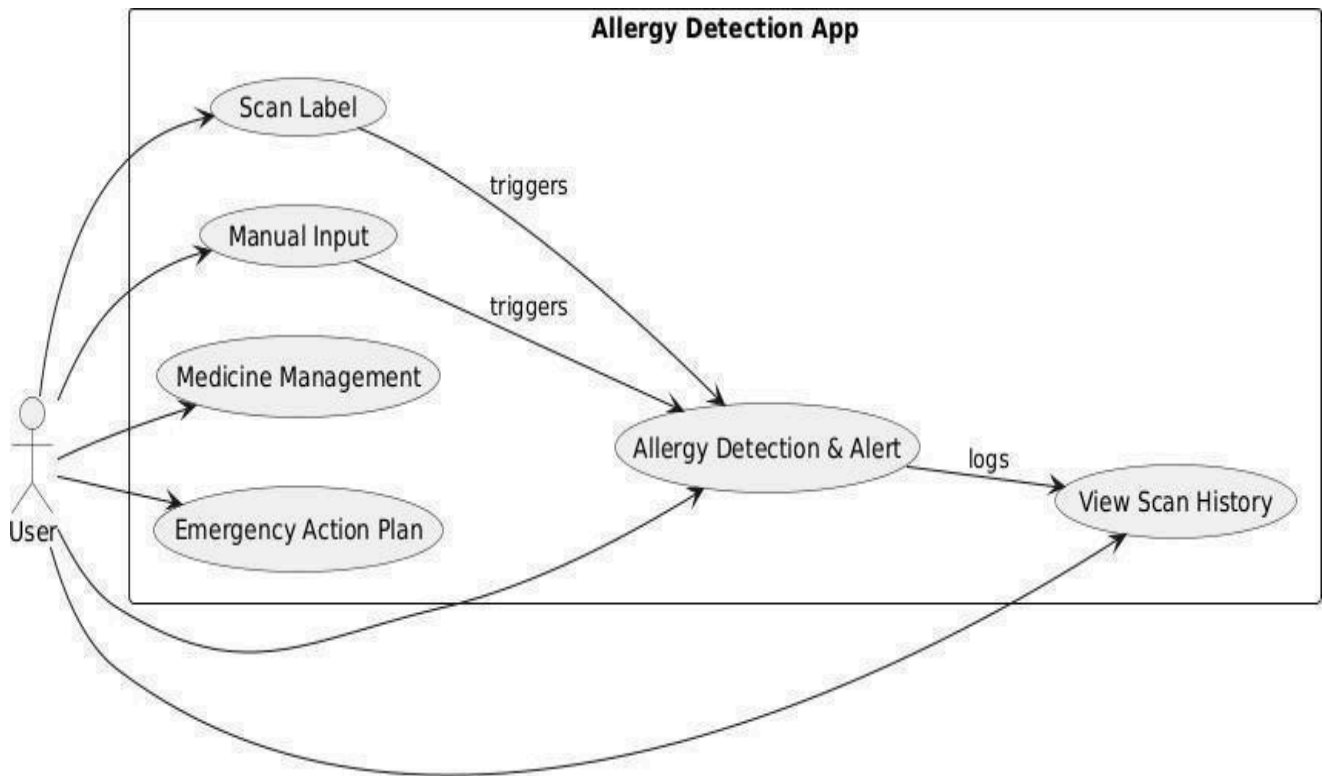
3.1.3.1 **Development Machines**:

3.1.3.1.1 At least 8GB RAM (for running local backend, database,).

3.1.3.1.2 Modern CPU for training ML models (TensorFlow/Scikit-learn).

Chapter 4: System Design

4.1 Use Case Diagram



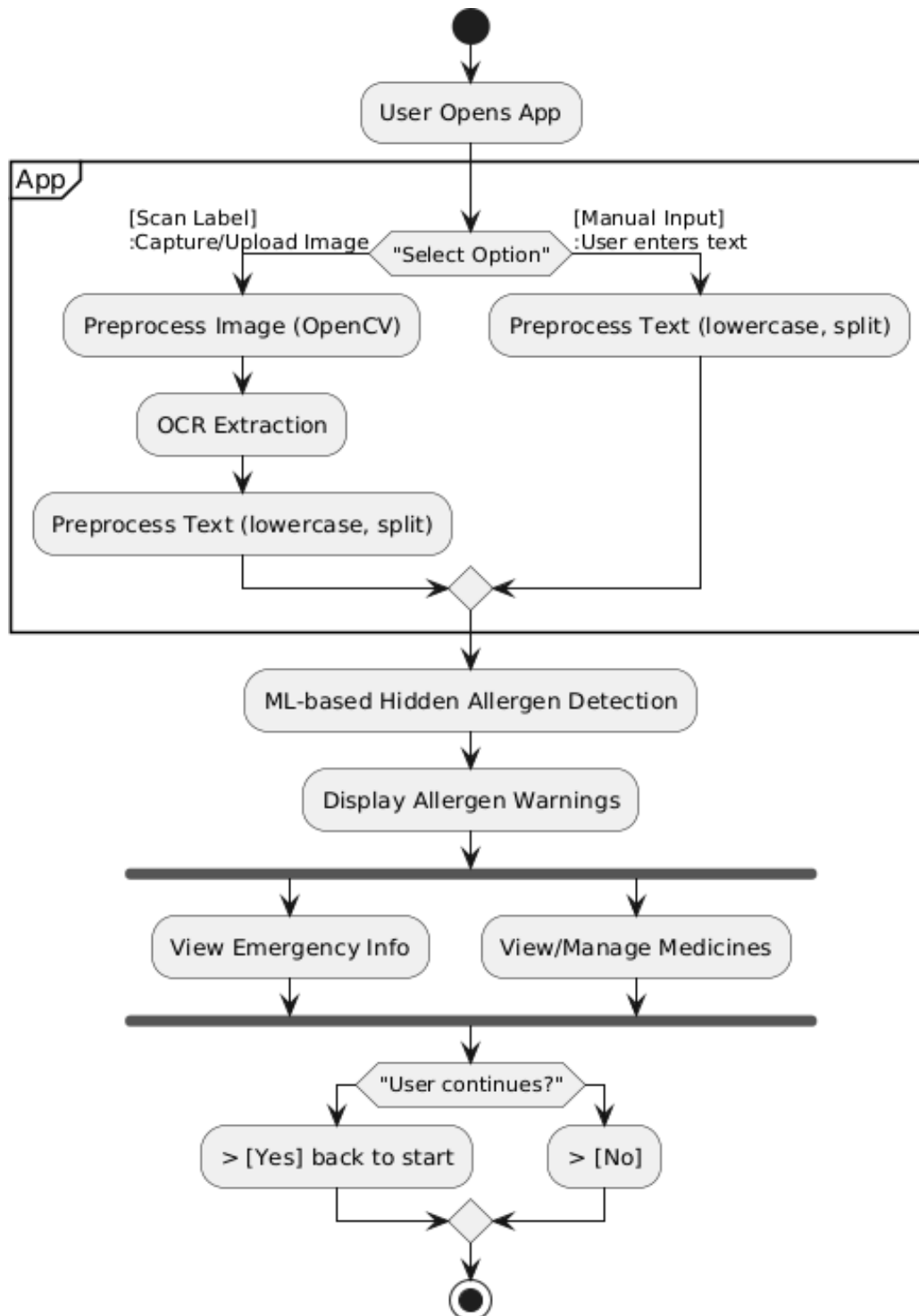
The use case diagram illustrates the functional interactions between the user and the Allergy Detection App. The central goal of the application is to detect potential allergens in food or product labels and assist the user in managing allergic reactions effectively.

The user can interact with the app in several ways. One way is by scanning a label using the device camera, which triggers the allergy detection and alert mechanism.

Alternatively, the user may choose to manually input ingredients or product names, which also triggers the allergy detection process. This core detection module compares the input data against a predefined list of allergens and, if any are found, alerts the user accordingly. All detection events are logged automatically, allowing the user to later access them through the view scan history feature.

In addition to detection, the app provides two more functional areas for proactive allergy management. The medicine management module allows users to keep track of allergy medications, dosages, and alerts for refills or expiry. The emergency action plan feature enables users to store predefined steps to follow during an allergic reaction, such as contact details for emergency services, instructions for using medication like epinephrine, and other personalized safety protocols. Altogether, the diagram represents a comprehensive allergy management system where scanning, manual input, and alerting are tightly integrated with historical tracking and emergency preparedness.

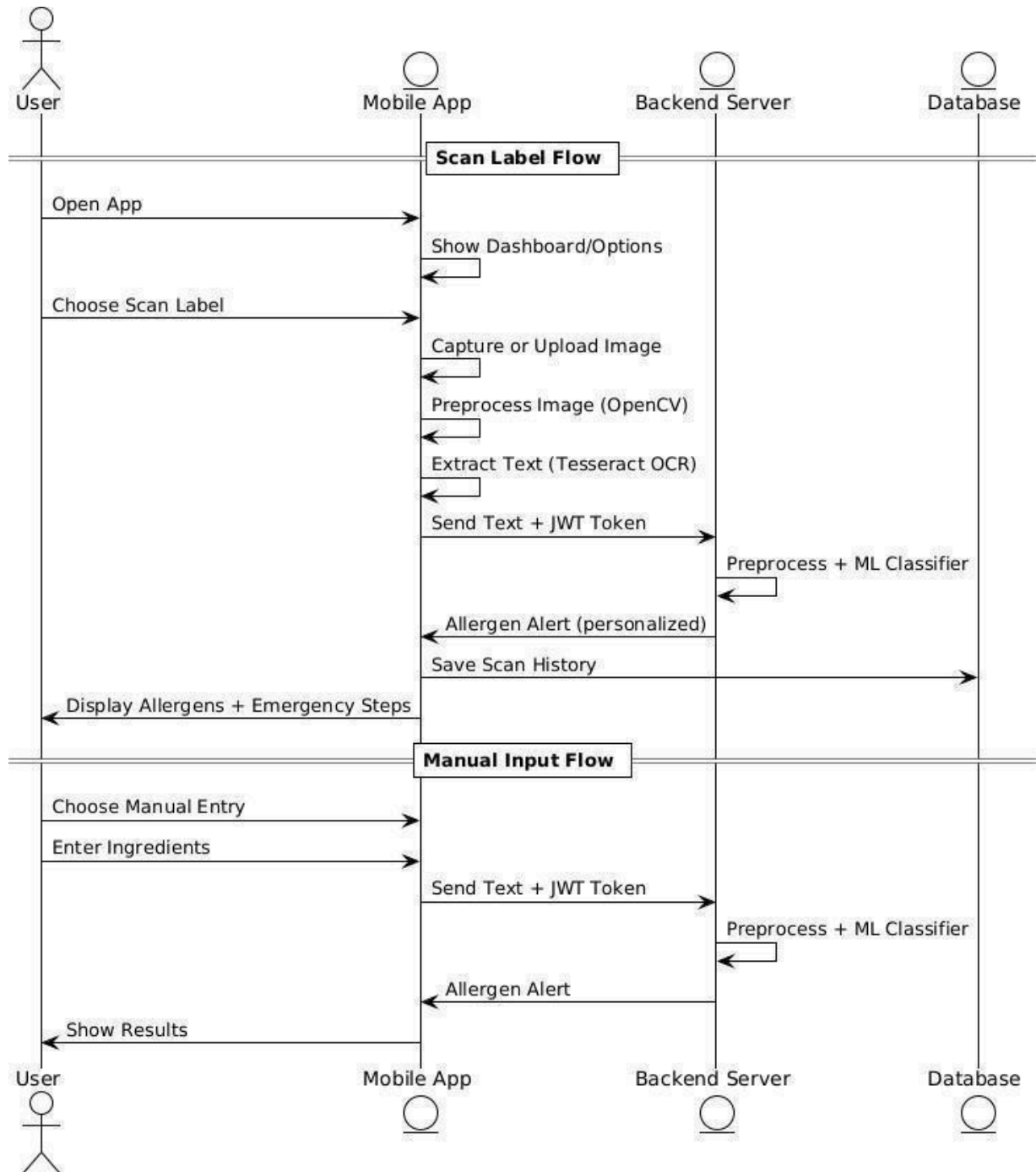
4.2 Activity Diagram



Activity Diagram – Explanation

- The activity begins when the user opens the app, which initiates the allergy detection workflow.
- The user is then presented with two input options:
 - Scan Label: The user can capture or upload an image of a product label.
 - Manual Input: The user can directly type in the list of ingredients or product content.
- If the scan option is selected:
 - The image is first preprocessed using OpenCV, typically to clean up noise and enhance text visibility.
 - Then, OCR extraction is applied to convert the image into raw text.
 - The extracted text is further preprocessed (converted to lowercase, split into individual words or tokens).
- If the manual input option is selected:
 - The entered text directly goes through the same preprocessing step (lowercase, split) to ensure consistency.
- After preprocessing, both paths converge into the ML-based hidden allergen detection module, which uses a trained machine learning model to analyze the text and detect any known or hidden allergens.
- If any allergens are detected, the app proceeds to display allergen warnings to alert the user.
- Following this, the user is provided access to two additional actions:
 - View Emergency Info: This displays personalized emergency procedures or contacts.
 - View/Manage Medicines: This allows the user to review or update allergy-related medications.
- Finally, the app checks whether the user wants to continue using the app.
 - If the user selects yes, the flow returns to the starting point to allow new input.
 - If the user selects no, the session ends and the activity concludes.
- This activity diagram captures the logical sequence of actions in the app and illustrates how user input is transformed into meaningful allergen alerts, followed by optional emergency and medicine-related utilities.

4.3 Sequence Diagram



Sequence Diagram – Explanation

Actors:

- User
- Mobile App
- Backend Server
- Database

Scan Label Flow:

- User opens app and selects "Scan Label" option.
- App captures or uploads an image.
- Image is preprocessed using OpenCV.
- Text is extracted using Tesseract OCR.
- Extracted text + JWT token is sent to the backend.
- Backend runs preprocessing and ML-based allergen detection.
- Allergen alert is sent back to the app.
- App displays allergen warning and emergency steps.
- Scan result is saved to the database.

Manual Input Flow:

- User selects "Manual Entry" and inputs ingredients.
- Text + JWT token is sent to the backend.
- Backend processes input and detects allergens via ML.
- Alert is returned to the app and shown to the user.

Note:

- Both flows share the same detection logic.
- JWT ensures secure communication.
- Database logs scan history for future access.

Chapter 5: Backend Implementation

This chapter details the backend implementation for our Allergy Detection system. It covers the directory structure, environment setup, and the primary Python modules.

5.1 Directory Structure

A simplified view of the backend's folder layout is shown below

```
allergy-app-backend/
├── api/
│   ├── endpoints/
│   │   ├── allergens.py
│   │   ├── auth.py
│   │   ├── ocr.py
│   │   └── scan_history.py
│   ├── test.py
│   ├── deps.py
│   └── routes.py
├── core/
│   ├── config.py
│   ├── constants.py
│   ├── database.py
│   └── security.py
├── models/
│   ├── scan_history.py
│   ├── schemas.py
│   └── user.py
├── services/
│   └── allergen_detector.py
├── label_binarizer.pkl
├── scripts/
│   ├── init_db.py
│   └── create_scan_history_table.py
```

```

├── main.py
├── .env
├── requirements.txt
└── setup.py

```

- **app/**: Contains all application logic, organized into subfolders for **API**, **core** functionalities (database, security, config), **models** (database models & schemas), and **services** (like `allergen_detector.py`).
- **data/**: May hold CSV or other data files used for training/testing.
- **model/**: Stores serialized artifacts (model and vectorizer) after training.
- **scripts/**: Utility scripts for initializing databases, training models, etc.
- **main.py**: The entry point for launching the FastAPI application.
- **.env**: Environment variables (e.g., database credentials, secrets).
- **requirements.txt**: Python dependencies needed by the project.

5.2 main.py

This is the **entry point** of our FastAPI application. It initializes the database tables, sets up CORS, and includes our master router.

```

1  from fastapi import FastAPI
2  from app.api.routes import router
3  from fastapi.middleware.cors import CORSMiddleware
4  from app.core.database import Base, engine
5  from dotenv import load_dotenv
6  import os
7
8  # Load environment variables from .env file
9  load_dotenv()
10
11 # Create database tables
12 Base.metadata.create_all(bind=engine)
13
14 app = FastAPI(title="Allergen Detection API")
15
16 # For development, allow all origins
17 # In production, you should be more restrictive
18 app.add_middleware(
19     CORSMiddleware,
20     allow_origins=["*"], # Allow all origins
21     allow_credentials=False, # Don't allow credentials with wildcard origins
22     allow_methods=["*"], # Allow all methods
23     allow_headers=["*"], # Allow all headers
24 )
25
26 app.include_router(router)

```

5.3 routes.py

Our **central router** that collects the endpoint routers defined in the **endpoints** folder.

```
from fastapi import APIRouter
from app.api.endpoints import allergens, ocr, auth, test, scan_history, medicines
router = APIRouter()

router.include_router(
    allergens.router,
    prefix="/allergens",
    tags=["allergens"]
)
router.include_router(
    ocr.router, prefix="", tags=["OCR"]
)
router.include_router(
    auth.router,
    prefix="/auth",
    tags=["authentication"]
)
router.include_router(
    test.router,
    prefix="/test",
    tags=["test"]
)
router.include_router(
    scan_history.router,
    prefix="/scan-history",
    tags=["scan-history"]
)
router.include_router(
    medicines.router,
    prefix="/medicines",
    tags=["medicines"]
)
```

5.4 deps.py

Houses **dependency functions** (e.g., extracting and validating tokens) that other endpoints rely on.

```
from fastapi import Depends, HTTPException, status, Header
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError
from sqlalchemy.orm import Session
from typing import Optional
from app.core.database import get_db
from app.core.security import SECRET_KEY, ALGORITHM
from app.models.schemas import TokenPayload
from app.models.user import User

# Configure OAuth2 with the correct token URL
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login", auto_error=False)

async def get_token_from_header(authorization: Optional[str] = Header(None)) -> Optional[str]:
    """
    Extract token from Authorization header.
    """
    if not authorization:
        return None

    try:
        scheme, token = authorization.split(" ", 1)
        if scheme.lower() != "bearer":
            return None

        if not token or not isinstance(token, str):
            return None

        return token.strip()
    except Exception:
        return None

async def get_current_user(
    db: Session = Depends(get_db),
    authorization: Optional[str] = Header(None)
) -> User:
    """
    Get the current user from the token.
    """
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
```



```

except JWTError as e:
    print(f"JWT Error: {str(e)}") # Add logging
    raise credentials_exception
except Exception as e:
    print(f"Unexpected error: {str(e)}") # Add logging
    raise credentials_exception

# Get the user from the database
user = db.query(User).filter(User.id == user_id).first()
if user is None:
    raise credentials_exception

return user

```

```

def get_current_user_optional(
    db: Session = Depends(get_db),
    token: str = Depends(oauth2_scheme)
) -> Optional[User]:
    """
    Get the current user from the token if it exists and is valid.
    Returns None if no token or token is invalid.
    """
    if not token:
        return None

    try:
        # Decode the token
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

        # Check if it's an access token
        if payload.get("type") != "access":
            return None
    
```

```

token = await get_token_from_header(authorization)
if not token:
    raise credentials_exception

try:
    # Decode the token
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

    # Check if it's an access token
    if payload.get("type") != "access":
        raise credentials_exception

    # Get the user ID from the token
    user_id_str: str = payload.get("sub")
    if user_id_str is None:
        raise credentials_exception

    # Convert string user_id to integer
    try:
        user_id = int(user_id_str)
    except ValueError:
        raise credentials_exception

except JWTError as e:
    print(f"JWT Error: {str(e)}") # Add logging
    raise credentials_exception
except Exception as e:
    print(f"Unexpected error: {str(e)}") # Add logging
    raise credentials_exception

# Get the user from the database
user = db.query(User).filter(User.id == user_id).first()
if user is None:
    raise credentials_exception

return user

```

```

def get_current_user_optional(
    db: Session = Depends(get_db),
    token: str = Depends(oauth2_scheme)
) -> Optional[User]:
    if not token:
        return None
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

        if payload.get("type") != "access":
            return None

        user_id_str: str = payload.get("sub")
        if user_id_str is None:
            return None

        try:
            user_id = int(user_id_str)
        except ValueError:
            return None

    except JWTError:
        return None
    user = db.query(User).filter(User.id == user_id).first()
    return user

def get_current_active_user(
    current_user: User = Depends(get_current_user),
) -> User:
    if not current_user.is_active:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Inactive user"
        )

    return current_user

```

5.5 API Endpoints

The `app/api/endpoints/` directory contains the actual REST endpoint implementations for allergen detection, user authentication, OCR, scan history, and general test/debug routes.

5.5.1 `allergens.py`

Purpose:

- 5.5.1.1 Provides an endpoint (`/allergens/detect`) to run the allergen detection on raw text.
- 5.5.1.2 Optionally marks allergens against a logged-in user's known allergies.

```
@router.post("/detect")
def detect_allergens(
    input_data: TextInput,
    current_user: Optional[User] = Depends(get_current_user_optional)
):
    result = detector.detect(input_data.text)
    if current_user and hasattr(current_user, 'allergies') and current_user.allergies:
        user_allergies = current_user.allergies
        user_allergy_ids = [allergy.get("id", "").lower() for allergy in user_allergies if isinstance(allergy, dict)]
        allergen_mapping = {
            "dairy": "Milk",
            "egg": "Eggs",
            "peanut": "Peanuts",
            "tree_nuts": "Nuts",
            "soy": "Soy",
            "wheat": "Wheat",
            "fish": "Fish",
            "shellfish": "Shellfish"
        }
        for allergen in result["allergens"]:
            allergen_normalized = allergen["allergen"].lower().replace(" ", "_")
            frontend_name = allergen_mapping.get(allergen_normalized, allergen["allergen"])

            if (frontend_name.lower() in user_allergy_ids or
                any(aid in frontend_name.lower() for aid in user_allergy_ids) or
                any(frontend_name.lower() in aid for aid in user_allergy_ids)):
                allergen["is_user_allergen"] = True
            else:
                allergen["is_user_allergen"] = False
        else:
            for allergen in result["allergens"]:
                allergen["is_user_allergen"] = False

    return result
```

Key Points:

- 5.5.1.3 Uses `AllergenDetector` (from `app/services/allergen_detector.py`) to classify allergens in text.
- 5.5.1.4 If a user is logged in, each detected allergen is flagged `is_user_allergen = True` if it matches their known allergy IDs.

5.5.2 auth.py

Purpose:

- 5.5.2.1 Handles user registration ([/auth/register](#)), login ([/auth/login](#)), and token refresh ([/auth/refresh](#)).
- 5.5.2.2 Includes an endpoint to get the current user's profile ([/auth/me](#)) and to update user allergies ([/auth/me/allergies](#)).

```
@router.post("/register", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
def register(user_in: UserCreate, db: Session = Depends(get_db)):
    """
    Register a new user.
    """
    user_by_email = db.query(User).filter(User.email == user_in.email).first()
    if user_by_email:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Email already registered"
        )

    user_by_username = db.query(User).filter(User.username == user_in.username).first()
    if user_by_username:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Username already taken"
        )

    hashed_password = get_password_hash(user_in.password)
    db_user = User(
        email=user_in.email,
        username=user_in.username,
        hashed_password=hashed_password
    )

    if hasattr(user_in, 'allergies') and user_in.allergies:
        db_user.allergies = [allergy.dict() for allergy in user_in.allergies]

    db.add(db_user)
    db.commit()
    db.refresh(db_user)

    return db_user
```

```

@router.get("/me", response_model=UserResponse)
def read_users_me(current_user: User = Depends(get_current_active_user), db: Session = Depends(get_db)):
    """
    Get current user.
    """
    # Make sure allergies is never None
    if not hasattr(current_user, 'allergies') or current_user.allergies is None:
        current_user.allergies = []
        db.commit()
        db.refresh(current_user)

    return current_user

@router.post("/refresh", response_model=Token)
def refresh_token(refresh_token_in: RefreshToken, db: Session = Depends(get_db)):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(refresh_token_in.refresh_token, SECRET_KEY, algorithms=[ALGORITHM])
        if payload.get("type") != "refresh":
            raise credentials_exception

        user_id: int = payload.get("sub")
        if user_id is None:
            raise credentials_exception

    except JWTError:
        raise credentials_exception

    user = db.query(User).filter(User.id == user_id).first()
    if user is None or not user.is_active:
        raise credentials_exception

    access_token = create_access_token(data={"sub": user.id})
    refresh_token = create_refresh_token(data={"sub": user.id})

    return {
        "access_token": access_token,
        "refresh_token": refresh_token,
        "token_type": "bearer"
    }

```

Key Points:

- **Registration** and **login** store hashed passwords (no plain text).
- **debug-auth** is a convenient endpoint for debugging JWT tokens.

5.5.3 ocr.py

Purpose:

- 5.5.3.1 Exposes an endpoint to handle **image file uploads** and run **OCR**
- 5.5.3.2 Preprocesses images (e.g., orientation correction, denoising, thresholding) to improve text extraction accuracy.

```
def correct_orientation(image: np.ndarray) -> np.ndarray:
    try:
        osd = pytesseract.image_to_osd(image)
        angle = int(re.search(r'Rotate: (\d+)', osd).group(1))
        if angle != 0:
            h, w = image.shape[:2]
            center = (w // 2, h // 2)
            M = cv2.getRotationMatrix2D(center, angle, 1.0)
            return cv2.warpAffine(image, M, (w, h), borderMode=cv2.BORDER_REPLICATE)
    except Exception as e:
        logger.warning(f"Orientation detection failed: {str(e)}")
    return image

def enhance_image(image: np.ndarray) -> np.ndarray:
    h, w = image.shape[:2]
    min_dim = 1800 # Minimum dimension for good OCR results

    if max(h, w) < min_dim:
        scale_factor = min_dim / max(h, w)
        return cv2.resize(image, None, fx=scale_factor, fy=scale_factor,
                           interpolation=cv2.INTER_CUBIC)
    return image
```

```

def create_processed_variants(image: np.ndarray) -> List[np.ndarray]:
    variants = []

    if len(image.shape) > 2:
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    else:
        gray = image.copy()

    variants.append(gray)

    denoised = cv2.fastNlMeansDenoising(gray, None, h=10, searchWindowSize=21, templateWindowSize=7)
    variants.append(denoised)

    kernel = np.array([[ -1, -1, -1], [ -1, 9, -1], [ -1, -1, -1]])
    sharpened = cv2.filter2D(denoised, -1, kernel)
    variants.append(sharpened)

    # 1. Otsu threshold
    _, otsu = cv2.threshold(sharpened, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    variants.append(otsu)

    # 2. Adaptive threshold
    adaptive = cv2.adaptiveThreshold(
        denoised, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2
    )
    variants.append(adaptive)

    # 3. Inverted version of thresholds to handle white-on-black text
    variants.append(cv2.bitwise_not(otsu))
    variants.append(cv2.bitwise_not(adaptive))

    return variants


def clean_text(text: str) -> str:
    # Remove excessive whitespace
    cleaned = re.sub(r'\s+', ' ', text).strip()

    # Fix common OCR errors
    cleaned = re.sub(r'\l\b', 'i', cleaned) # Fix 'l' at end of words to 'i'
    cleaned = re.sub(r'0', '0', cleaned) # Fix '0' to '0' in numbers
    cleaned = re.sub(r'(\d),(\d)', r'\1.\2', cleaned) # Fix comma to decimal in numbers

    # Fix spacing around punctuation
    cleaned = re.sub(r'\s+([.,;:)]', r'\1', cleaned)
    cleaned = re.sub(r'([([)])\s+', r'\1', cleaned)

    # Fix sentence boundaries
    cleaned = re.sub(r'([a-z])\.([A-Z])', r'\1. \2', cleaned)

    return cleaned

```



```

@router.post("/ocr")
async def high_quality_ocr(
    file: Annotated[UploadFile, File()],
    dpi: int = 300,
    language: str = "eng",
    psm: Optional[int] = None
):
    try:
        # Validate input
        if not file.content_type.startswith('image/'):
            raise HTTPException(400, "Invalid file type")

        contents = await file.read()
        if not contents:
            raise HTTPException(400, "Empty file received")

        np_array = np.frombuffer(contents, np.uint8)
        image = cv2.imdecode(np_array, cv2.IMREAD_COLOR)
        if image is None:
            raise HTTPException(400, "Invalid image format")
        corrected = correct_orientation(image)
        enhanced = enhance_image(corrected)
        variants = create_processed_variants(enhanced)
        if psm is None:
            h, w = enhanced.shape[:2]
            if max(h, w) > 1200: # Larger images likely have multiple text blocks
                psm = 3 # Multiple columns/paragraphs
            elif h < 200 or w / h > 5: # Very narrow image likely a single line
                psm = 7 # Single text line
            else:
                psm = 11 # Sparse text
        custom_config = (
            f"--dpi {dpi} "
            f"--psm {psm} "
            "--oem 3 "

```

```

        f"-l {language} "
        "-c tesseract_char_whitelist=\"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-.,;:/%'\\\\\"
    )
    all_texts = []
    for variant in variants:
        text = pytesseract.image_to_string(variant, config=custom_config).strip()
        if text:
            all_texts.append(text)
    if not all_texts:
        return {"text": "", "success": False, "message": "No text detected in image"}
    longest_text = max(all_texts, key=len)
    cleaned_text = clean_text(longest_text)
    if len(all_texts) > 1:
        words = {}
        for text in all_texts:
            for word in re.findall(r'\b\w+\b', text.lower()):
                if len(word) > 2: # Only count words with 3+ characters
                    words[word] = words.get(word, 0) + 1

        common_words = {word for word, count in words.items() if count > 1}
        longest_words = set(re.findall(r'\b\w+\b', longest_text.lower()))

        if len(common_words) > 0 and len(common_words - longest_words) > len(common_words) * 0.3:
            # Combine all texts and clean
            combined_text = ' '.join(all_texts)
            cleaned_text = clean_text(combined_text)

    return {"text": cleaned_text, "success": True}

except HTTPException:
    raise
except Exception as e:
    logger.error(f"OCR Error: {str(e)}", exc_info=True)
    raise HTTPException(500, f"Processing error: {str(e)}")

```

Key Points:

- Uses **OpenCV** to handle image transformations.
- Tesseract config is dynamically set to choose the right Page Segmentation Mode (PSM).

5.5.4 scan_history.py

```
@router.post("/", response_model=ScanHistoryResponse)
def create_scan_history(
    scan_data: ScanHistoryCreate,
    db: Session = Depends(get_db),
    current_user = Depends(get_current_user_optional)
):
    print(f"Received scan data: {scan_data}")
    user_id = current_user.id if current_user else None
    if user_id and not scan_data.user_id:
        scan_data.user_id = user_id
    try:
        db_scan = ScanHistory(
            user_id=scan_data.user_id,
            product_name=scan_data.product_name,
            input_text=scan_data.input_text,
            allergens=scan_data.allergens,
            image_url=scan_data.image_url
        )
        db.add(db_scan)
        db.commit()
        db.refresh(db_scan)
        print(f"Created scan history with ID: {db_scan.id}")
        response = {
            "id": db_scan.id,
            "user_id": db_scan.user_id,
            "product_name": db_scan.product_name,
            "input_text": db_scan.input_text,
            "allergens": db_scan.allergens,
            "image_url": db_scan.image_url,
            "created_at": db_scan.created_at
        }
        return response
    except Exception as e:
        import traceback
        print(f"Error creating scan history: {e}")
        print(traceback.format_exc())
        raise HTTPException(status_code=500, detail=f"Error creating scan history: {str(e)}")
```

```

@router.get("/", response_model=List[ScanHistoryResponse])
def get_scan_history(
    skip: int = 0,
    limit: int = 100,
    db: Session = Depends(get_db),
    current_user = Depends(get_current_user_optional)
):
    """Get scan history for the current user."""
    # If not logged in, return only device-specific entries (no user_id)
    if not current_user:
        query = db.query(ScanHistory).filter(ScanHistory.user_id.is_(None))
    else:
        # If logged in, return the user's entries
        query = db.query(ScanHistory).filter(ScanHistory.user_id == current_user.id)

    results = query.order_by(ScanHistory.created_at.desc()).offset(skip).limit(limit).all()

    # Convert database results to response format
    response_items = []
    for item in results:
        response_item = {
            "id": item.id,
            "user_id": item.user_id,
            "product_name": item.product_name,
            "input_text": item.input_text,
            "allergens": item.allergens, # This should already be a list of dicts
            "image_url": item.image_url,
            "created_at": item.created_at
        }
        response_items.append(response_item)

    return response_items

```

Key Points:

- `current_user` is optional, allowing anonymous scans.
- Restricts access so a user can only view/delete **their own** scans.

5.5.5 test.py

Purpose:

- 5.5.5.1 Contains simple test endpoints (e.g., `/test/`) and a debug-auth endpoint to examine request headers and tokens.

Key Points:

- 5.5.5.2 Useful for verifying the system is up and running without requiring user credentials.
- 5.5.5.3 Demonstrates how to decode a JWT from an `Authorization` header.

5.5.6 routes_assembly.py

After defining each endpoint in its own file, they're assembled in routes.py

```
from fastapi import APIRouter
from app.api.endpoints import allergens, ocr, auth, test, scan_history, medicines
router = APIRouter()

router.include_router(
    allergens.router,
    prefix="/allergens",
    tags=["allergens"]
)
router.include_router(
    ocr.router, prefix="", tags=["OCR"]
)
router.include_router(
    auth.router,
    prefix="/auth",
    tags=["authentication"]
)
router.include_router(
    test.router,
    prefix="/test",
    tags=["test"]
)
router.include_router(
    scan_history.router,
    prefix="/scan-history",
    tags=["scan-history"]
)
router.include_router(
    medicines.router,
    prefix="/medicines",
    tags=["medicines"]
)
```

5.6 Core Files

These files under `app/core` handle **configuration**, **database setup**, **security** (JWT, password hashing), etc.

5.6.1 database.py

Sets up **SQLAlchemy** with PostgreSQL, providing a session via `get_db()`.

```

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
import os

user = os.getenv("POSTGRES_USER", "harismac")
password = os.getenv("POSTGRES_PASSWORD", "")
host = os.getenv("POSTGRES_HOST", "localhost")
port = os.getenv("POSTGRES_PORT", "5432")
db = os.getenv("POSTGRES_DB", "allergen_app")

if password:
    DATABASE_URL = f"postgresql://{user}:{password}@{host}:{port}/{db}"
else:
    DATABASE_URL = f"postgresql://{user}@{host}:{port}/{db}"

DATABASE_URL = os.getenv("DATABASE_URL", DATABASE_URL)

engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

5.6.2 Secruity.py

These files under `app/security.py` handle password verification , refresh token management , creation , access token etc .

```

from jose import JWTError, jwt
from passlib.context import CryptContext
import os

SECRET_KEY = os.getenv("SECRET_KEY", "09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 60 * 24 * 30)) # Default: 30 days
REFRESH_TOKEN_EXPIRE_MINUTES = int(os.getenv("REFRESH_TOKEN_EXPIRE_MINUTES", 60 * 24 * 60)) # Default: 60 days
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def verify_password(plain_password, hashed_password):
    return pwd_context.verify(plain_password, hashed_password)
def get_password_hash(password):
    return pwd_context.hash(password)
def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    if "sub" in to_encode and to_encode["sub"] is not None:
        to_encode["sub"] = str(to_encode["sub"])
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire, "type": "access"})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
def create_refresh_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    if "sub" in to_encode and to_encode["sub"] is not None:
        to_encode["sub"] = str(to_encode["sub"])
    if expires_delta:
        expire = datetime.utcnow() + expires_delta
    else:
        expire = datetime.utcnow() + timedelta(minutes=REFRESH_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire, "type": "refresh"})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

```

5.7 Services

5.7.1 allergen_detector.py

Loads and applies the **ML model** (trained offline) to detect allergen

```
import joblib
from app.core.config import MODEL_PATH, VECTORIZER_PATH, LABEL_BINARIZER_PATH
from app.core.constants import INGREDIENTS
from app.utils.text_processing import get_evidence

class AllergenDetector:
    def __init__(self):
        self.model = joblib.load(MODEL_PATH)
        self.tfidf = joblib.load(VECTORIZER_PATH)
        self.mlb = joblib.load(LABEL_BINARIZER_PATH)

        # Adjusted thresholds - lowered for better detection of direct ingredients
        self.primary_threshold = 0.35 # For direct ingredients
        self.secondary_threshold = 0.25 # For "may contain" statements

    def detect(self, text: str) -> dict:
        X = self.tfidf.transform([text])
        predictions_proba = self.model.predict_proba(X)

        # Use instance thresholds
        primary_threshold = self.primary_threshold
        secondary_threshold = self.secondary_threshold

        allergen_predictions = []
        for idx, label in enumerate(self.mlb.classes_):
            if not label:
                continue

            prob = predictions_proba[idx][0][1]
            evidence = get_evidence(text, label)

            if evidence:
                # Direct ingredients or "Contains" statements
                if any("Contains statement:" in e for e in evidence) or any(term.lower() in text.lower()
                                                                           for term in INGREDIENTS[label]):
```



```

        # Increase confidence for direct ingredient mentions
        prob = min(prob * 1.5, 1.0) # Increased from 1.2 to 1.5
        if prob >= primary_threshold:
            allergen_predictions.append({
                "allergen": label,
                "confidence": float(prob),
                "evidence": evidence
            })
        # "May contain" statements
        elif any("may contain" in e.lower() or "traces of" in e.lower() for e in evidence):
            if prob >= secondary_threshold:
                allergen_predictions.append({
                    "allergen": label,
                    "confidence": float(prob),
                    "evidence": evidence
                })

    return {
        "allergens": sorted(allergen_predictions,
                            key=lambda x: x["confidence"],
                            reverse=True),
        "input_text": text,
        "threshold_used": primary_threshold
    }

```

```

def detect_allergens(self, text: str) -> list[dict]:
    allergens = []
    predictions = self.model.predict_proba(self.tfidf.transform([text]))[0]

    for idx, confidence in enumerate(predictions):
        allergen = self.mlb.classes_[idx]
        evidence = get_evidence(text, allergen)

        # Only include high confidence predictions with evidence
        if evidence and confidence >= self.primary_threshold:
            allergens.append({
                "allergen": allergen,
                "confidence": float(confidence),
                "evidence": evidence
            })
        # Include lower confidence predictions only for "may contain" statements
        elif evidence and "may contain" in str(evidence).lower() and confidence >= self.secondary_threshold:
            allergens.append({
                "allergen": allergen,
                "confidence": float(confidence),
                "evidence": evidence
            })

    return sorted(allergens, key=lambda x: x["confidence"], reverse=True)

```

5.8 Model Training Process

This section details the **collection**, **preprocessing**, and **training** of the machine-learning model used for allergen detection. The goal is to identify specific allergen-related keywords, phrases, or hidden ingredients from scanned or manually entered text.

5.8.1 Data Collection and Sources

The allergen dataset used in this project was compiled from:

1. **Kaggle**: Various publicly available ingredient/allergen repositories, focusing on **multi-label** sets that list multiple allergens per entry.
2. **Open-Source Projects / Public Datasets**: Additional CSV files containing ingredient labels for food products, sometimes annotated with potential allergens.
3. **Custom Data Curation**: A smaller subset of manually labeled ingredient texts from packaging examples, aiming to capture region-specific or lesser-known allergens.

By merging these datasets, we aimed for **broad coverage** of common allergens (milk, egg, peanut, soy, etc.) and lesser-known terms (e.g., “casein,” “albumin,” “maltodextrin”).

5.8.2 Data Preprocessing

1. **Text Normalization**:
 - Converted ingredient descriptions to lowercase.
 - Removed punctuation and excessive whitespace.
 - Filtered out non-informative tokens (e.g., stop words, repeated filler text).
2. **Label Transformation**:
 - Each sample could have **multiple allergens** (milk, peanut, soy, etc.).
 - We used **MultiLabelBinarizer** to convert comma-separated allergen labels into multi-hot vectors (e.g., `[1,0,1]` if sample has milk & soy but not egg).
3. **Train/Test Split**:
 - Reserved **20%** of the data as a test set to evaluate final model performance.
 - The remaining **80%** was used for training, ensuring all major allergen classes appeared in both splits.

5.8.3 Model Choice and Rationale

5.8.3.1 Random Forest + Multi-Label Classification

- 5.8.3.1.1 We adopted a `RandomForestClassifier` within a `MultiOutputClassifier` wrapper to handle the multi-label nature of allergen detection (i.e., each record can have multiple allergen labels).

- 5.8.3.1.2 Random Forest was chosen for its robustness against overfitting, capacity to handle class imbalance (through `class_weight='balanced'`), and relatively straightforward tuning.
- 5.8.3.2 TF-IDF Vectorization
 - 5.8.3.2.1 We employed a `TfidfVectorizer` to capture term frequency–inverse document frequency signals.
 - 5.8.3.2.2 Using n-grams up to size 3 (`ngram_range=(1,3)`) helped detect short phrases (e.g., “milk powder,” “may contain,” etc.).
 - 5.8.3.2.3 Max features was set to around 3000 to avoid overly large feature spaces, while `min_df=2` and `max_df=0.95` filtered extremely rare or overly common terms.

5.8.4 Model Training Steps

1. Data Loading:

- The merged CSV files were placed in the `data/` folder under filenames like `allergen_dataset.csv`.
- Each row contained a text field (`ingredient_text`) and a set of allergens (comma-separated).

2. Vectorization and Label Binarization:

- Applied TF-IDF to transform `ingredient_text` into a sparse matrix of features.
- Used `MultiLabelBinarizer` to encode allergens into multi-hot arrays.

3. Random Forest Training:

- Wrapped in `MultiOutputClassifier` so each allergen label is predicted simultaneously.
- Key hyperparameters:
 - `n_estimators=200` for stable ensemble predictions.
 - `max_depth=10–15` to limit overfitting.
 - `min_samples_split=5` and `min_samples_leaf=2` for further regularization.
 - `class_weight='balanced'` to help with underrepresented allergens.

4. Evaluation:

- Computed **Hamming Loss**, which measures the fraction of wrong labels to total labels.
- Generated **precision**, **recall**, and **F1 scores** for each allergen label individually.
- Used different **decision thresholds** (0.3, 0.5, 0.7) to evaluate how many positives or negatives the model returns under varying confidence levels.

5. Model Serialization:

- Saved the final classifier (`allergen_model.pkl`), TF-IDF vectorizer (`tfidf_vectorizer.pkl`), and label binarizer (`label_binarizer.pkl`) using `joblib`.
- These `.pkl` files are loaded in `allergen_detector.py` during inference.

model_training.py

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import
TfidfVectorizer from sklearn.multioutput import
MultiOutputClassifier from sklearn.ensemble import
RandomForestClassifier
from sklearn.metrics import classification_report, hamming_loss
import joblib
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import precision_score

# Convert comma-separated string labels to lists
return [set(l.split(',')) for l in labels]

def create_model():
    # Create text processing pipeline
    text_pipeline = Pipeline([ ('tfidf',
        TfidfVectorizer(
            ngram_range=(1,
                3),
            max_features=3000,
            min_df=2,
            max_df=0.95,
            analyzer='word',
            token_pattern=r'(?u)\b\w+\b|([^\s]*)\b\w+\s+\w+\b'
        )),
    ])

    # Create classifier pipeline
    clf_pipeline = Pipeline([
        ('rf', RandomForestClassifier(
            n_estimators=200, max_depth=15,
```

```

        min_samples_split=5,
        min_samples_leaf=2,
        class_weight='balanced',
        random_state=42
    ))
])

return Pipeline([
    ('text', text_pipeline),
    ('clf', MultiOutputClassifier(clf_pipeline))
])

def train_model():
    # Load and preprocess data
    df = pd.read_csv("data/allergen_dataset.csv")

    # Split data
    X_train, X_test, y_train, y_test = train_test_split(
        df['ingredient_text'], df['allergens'].str.get_dummies(sep=' '),
        test_size=0.2,
        random_state=42
    )

    # Create and train model
    model = create_model()
    model.fit(X_train,
              y_train)

    # Evaluate
    y_pred_proba = model.predict_proba(X_test)
    for i, label in enumerate(y_train.columns): print(f"\nMetrics for
        {label}:")
        proba = y_pred_proba[i][:, 1]
        for threshold in [0.3, 0.5, 0.7]:
            pred = (proba >= threshold).astype(int)
            print(f"Precision at {threshold}:
{precision_score(y_test.iloc[:, i], pred):.3f}")

def main():
    df = pd.read_csv("data/allergen_dataset.csv") labels =
    preprocess_labels(df['allergens'])

    # MultiLabelBinarizer
    mlb = MultiLabelBinarizer() y

```

```

= mlb.fit_transform(labels)

tfidf =
    TfidfVectorizer(
        ngram_range=(1, 3),
        max_features=3000,
        min_df=2,
        max_df=0.95,
        stop_words='english',
        token_pattern=r'(?u)\b\w+\b|\'([^\']*)\'',
        strip_accents='unicode'
    )
X = tfidf.fit_transform(df['ingredient_text'])

X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42, shuffle=True
)

base_classifier = RandomForestClassifier(n_estimators=200,
    max_depth=10,
    min_samples_split=5,

    min_samples_leaf=2,
    max_features='sqrt',
    class_weight='balanced',
    n_jobs=-1,
    random_state=42
)

model = MultiOutputClassifier(base_classifier, n_jobs=-1)
model.fit(X_train, y_train)

# Evaluate
y_pred_train = model.predict(X_train)
y_pred_test
= model.predict(X_test)

print("Training Metrics:")
print(f"Hamming Loss: {hamming_loss(y_train,
y_pred_train):.4f}")
print("Test Metrics:")
print(f"Hamming Loss: {hamming_loss(y_test, y_pred_test):.4f}")

print("\nDetailed Classification Report:")
for i, allergen in enumerate(mlb.classes_):

```

```

    print(f"\nMetrics for {allergen}:")
    print(classification_report(y_test[:, i], y_pred_test[:, i], target_names=['No', 'Yes']))

# Top features
feature_names = tfidf.get_feature_names_out()
for i, allergen in enumerate(mlb.classes_):
    importances = model.estimators_[i].feature_importances_
    top_features = sorted(zip(importances, feature_names),
reverse=True)[:10]
    print(f"\nTop 10 important features for {allergen}:")
    for importance, feature in top_features: print(f"{feature}:
        {importance:.4f}")

# Save the model artifacts
joblib.dump(model,
"model/allergen_model.pkl")
joblib.dump(tfidf,
"model/tfidf_vectorizer.pkl")
joblib.dump(mlb,
"model/label_binarizer.pkl")

if __name__ == "__main__":
    main()

```

Chapter 6: Frontend Overview

This chapter provides a concise overview of our **mobile frontend**, built with **React Native** (via **Expo**). The goal is to enable users to **register** or **log in**, **scan** product labels for allergens, **manually enter** ingredients, and **manage medicines**, all while communicating seamlessly with our FastAPI backend.

6.1 Technology Stack and Framework

1. React Native (Expo)

- Chosen for cross-platform compatibility (Android/iOS) and rapid development.
- Expo handles over-the-air updates, easy testing, and hardware integrations (camera for scanning).

2. React Navigation / Expo Router

- Provides **file-based routing** or standard navigation flows (tabs, stacks).
- Separates screens into logical folders:
 - `/(auth)/` for login/signup screens
 - `/(tabs)/` for main app functionalities (scan, medicine, etc.)

3. API Integration

- Makes simple **fetch** requests to our backend endpoints (`/auth/login`, `/ocr`, `/allergens/detect`, etc.).
 - An `API_URL` constant helps keep the base URL consistent across different environments (development vs. production).
-

6.2 Screen Flows

6.2.1 Auth Screens (Sign Up & Login)

6.2.1.1 **Purpose:** Allow new users to create an account and returning users to sign in.

6.2.1.2 **Layout:** Simple forms with fields like **username**, **email**, **password**, and **confirm password**.

6.2.1.3 **Interaction with Backend:**

- 6.2.1.3.1 **Sign Up:** Calls [POST /auth/register](#) with user info.
- 6.2.1.3.2 **Login:** Calls [POST /auth/login](#) to receive JWT tokens (access and refresh).
- 6.2.1.3.3 **Error Handling:** Displays alerts if invalid credentials or server issues occur.

6.2.2 Home Screen

- 6.2.2.1 **Highlights:** Quick overview of features and recent activity.
- 6.2.2.2 **Quick Actions:**
 - 6.2.2.2.1 **Scan Products** (launches camera-based OCR screen).
 - 6.2.2.2.2 **Manual Entry** (directly type ingredients).
 - 6.2.2.2.3 **Emergency Info** (emergency contacts/instructions).
 - 6.2.2.2.4 **Medicine Tracker** (list of user's medications, set reminders).
- 6.2.2.3 **Recent Activity:** Summaries of past scans or manual checks (fetched from [/scan-history](#)).

6.2.3 Scan Screen

- 6.2.3.1 **Goal:** Let users capture a product label image or select one from the gallery.
- 6.2.3.2 **Camera & Gallery:** Utilizes Expo's [ImagePicker](#) or direct camera permissions.
- 6.2.3.3 **OCR Processing:**
 - 6.2.3.3.1 The image is sent to [POST /ocr](#) on the backend.
 - 6.2.3.3.2 Returns extracted text, which can then be analyzed via the allergen detection endpoint.
- 6.2.3.4 **UI:**
 - 6.2.3.4.1 Large preview area for the picked photo.
 - 6.2.3.4.2 Buttons for "Take Photo," "Gallery," and "Scan Label."

6.2.4 Results / Allergen Detection

- 6.2.4.1 **After OCR:** The extracted text can be displayed to the user.
- 6.2.4.2 **Allergen Check:** Optionally calls [POST /allergens/detect](#) to highlight potential allergens.
- 6.2.4.3 **UI:**
 - 6.2.4.3.1 Shows identified allergens with confidence scores.
 - 6.2.4.3.2 If the user is logged in and has saved allergy preferences, flags any matching ones.

6.2.5 Manual Entry Screen

- 6.2.5.1 **Purpose:** For scenarios where OCR is not possible or the label is text-based.
- 6.2.5.2 **Process:** User inputs or pastes ingredient text; app sends it to [/allergens/detect](#).
- 6.2.5.3 **UI:** A simple text field with a "Check Allergens" button.

6.2.6 Medicine Tracker

6.2.6.1 Goals:

6.2.6.1.1 Allow users to store a list of medications (e.g., EpiPen, antihistamines) with dosages and expiration dates.

6.2.6.1.2 Possibly display alerts for soon-to-expire meds.

6.2.6.2 **API Calls:** Could be **GET /medicines** and **POST /medicines** if the backend supports it.

6.2.6.3 UI:

6.2.6.3.1 List of existing meds.

6.2.6.3.2 Fields to add new medication name, dosage, etc.

6.2.7 Emergency Info Screen

6.2.7.1 **Details:** Offers immediate instructions or contact information in an allergic reaction scenario.

6.2.7.2 **Static or Configurable:** Could be a static resource, or user can customize emergency contacts.

6.3 Navigation and Layout

- **Tabbed Navigation:**
 - Typical bottom tab bar includes “Home,” “Scan,” “Manual,” “Medicine,” “Emergency,” etc.
 - Each tab corresponds to a distinct user flow.
 - **Auth Flow:**
 - If a user is not authenticated, the app automatically navigates to the **Login** or **Signup** screens.
 - On successful login, the user is directed to the main tab screens.
 - **Dark Mode:**
 - The app can use React Native’s appearance or custom styling for a **dark theme**, as the screenshots show a dark UI.
-

6.4 Styling and Components

1. **UI Components:**
 - **Custom Buttons:** Common button component for sign-ups, logins, scanning triggers, etc.
 - **Input Fields:** Used for both email/password forms and manual ingredient entry.
2. **Styling Approach:**
 - Tailwind CSS in React Native via something like **nativewind**.

- Dark mode support with color classes for text, backgrounds, etc.
-

6.5 User Experience Flow

1. **Launch App** → Sees **Login** or **Signup** if not authenticated.
2. **Home Screen** → Provides quick actions and recent scan activity.
3. **Choose Action**: e.g., “Scan Products.”
4. **Camera / Gallery** → Captures label → Sends to backend → Returns text →
Option to detect allergens.
5. **View Results** → Allergen highlights, advice on next steps.
6. **Add or View Medicines** → Track personal meds, store dosage info.
7. **Emergency Info** → Access contacts or guidelines if a reaction occurs.

This flow ensures an easy, intuitive experience. The user’s stored tokens handle authentication automatically, so repeated actions (like scanning multiple times) don’t prompt for login unless the token expires.

Chapter 7: Screenshots

This chapter provides a pictorial representation of our mobile app frontend, built with React Native (via Expo). It includes various pages including User Register and Log in Screen, Users Home Page, OCR Screen to scan product labels for allergens, Manual Entry Screen to manually enter ingredients, and Manage Medicines to store all the prescribed medicines for the user. There are also the output screen which provides the output of the OCR scan and list the allergens detected in the Result screen.

7.1 Log In Page

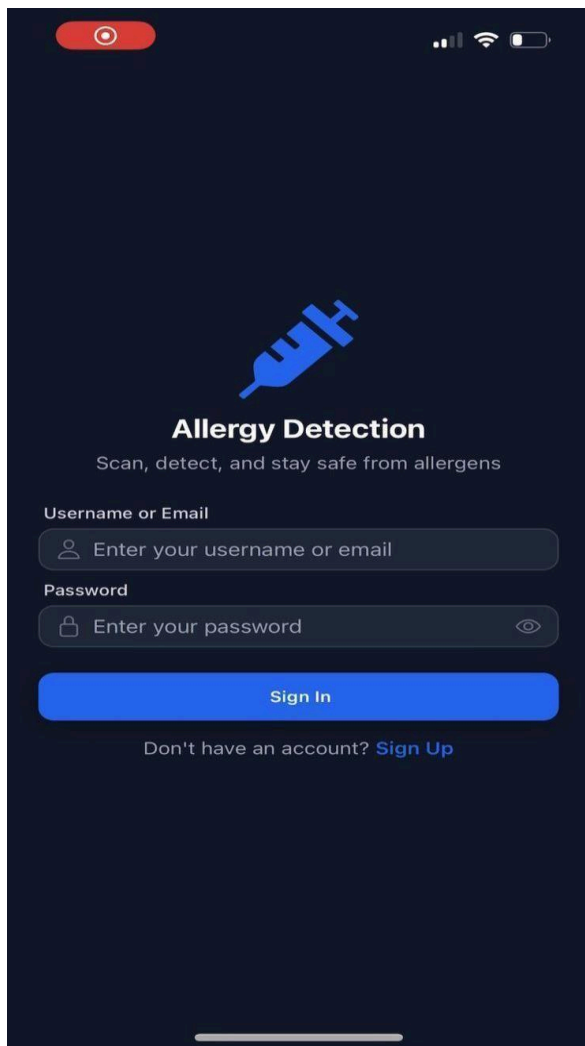


Fig 7.1: Users Register or Login Page

7.2 : Home Page

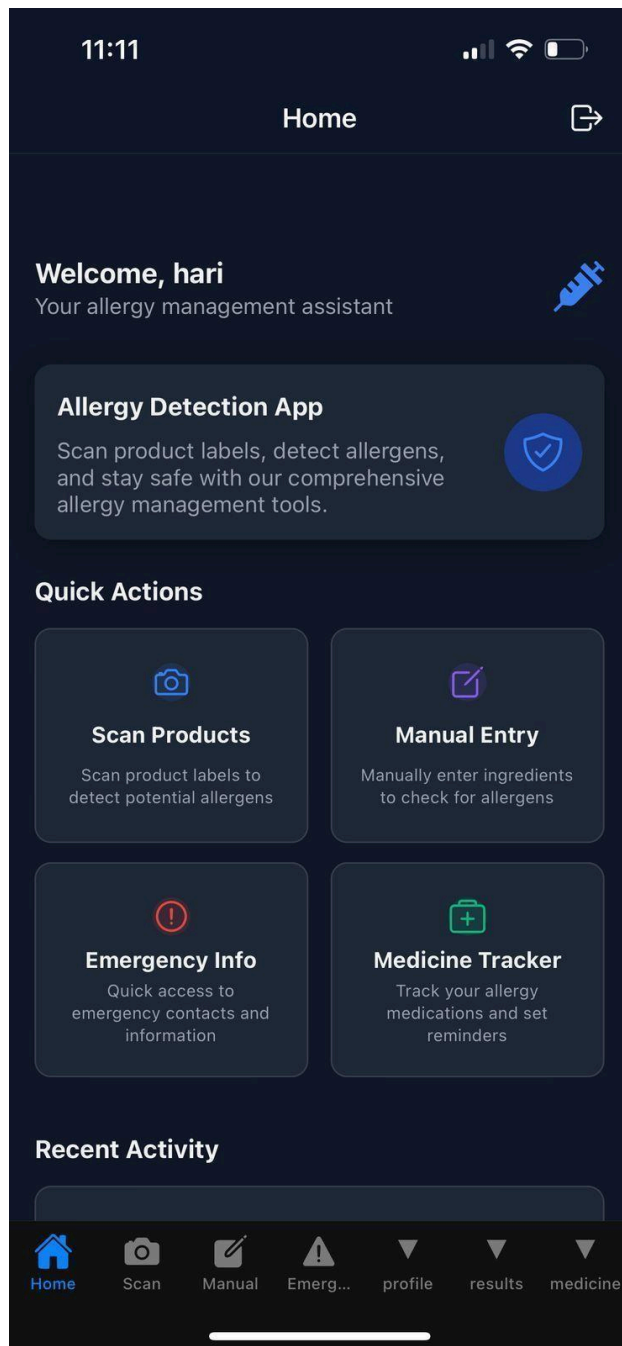


Fig 7.2: Shows the User Specific Home Page

7.3 : OCR Scan Product Label Page

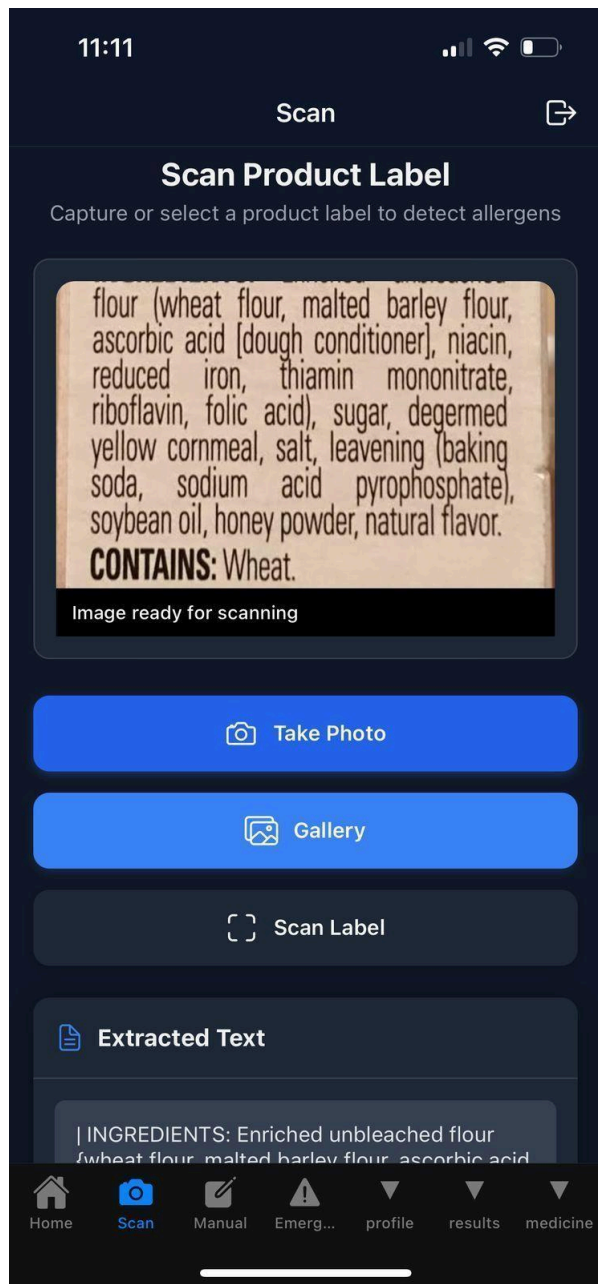


Fig 7.3: Scanning of a product label

7.4 : Scan Results Page

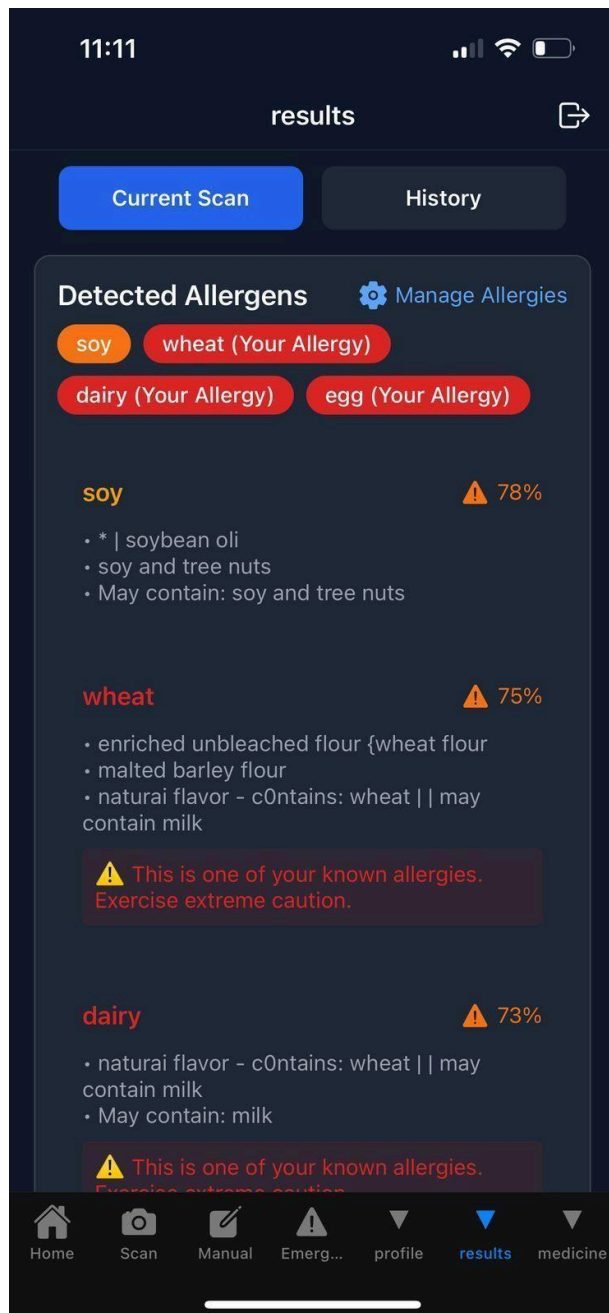


Fig 7.4: Shows the results of the OCR scan and the list of Allergens detected

7.5 : User Profile Page

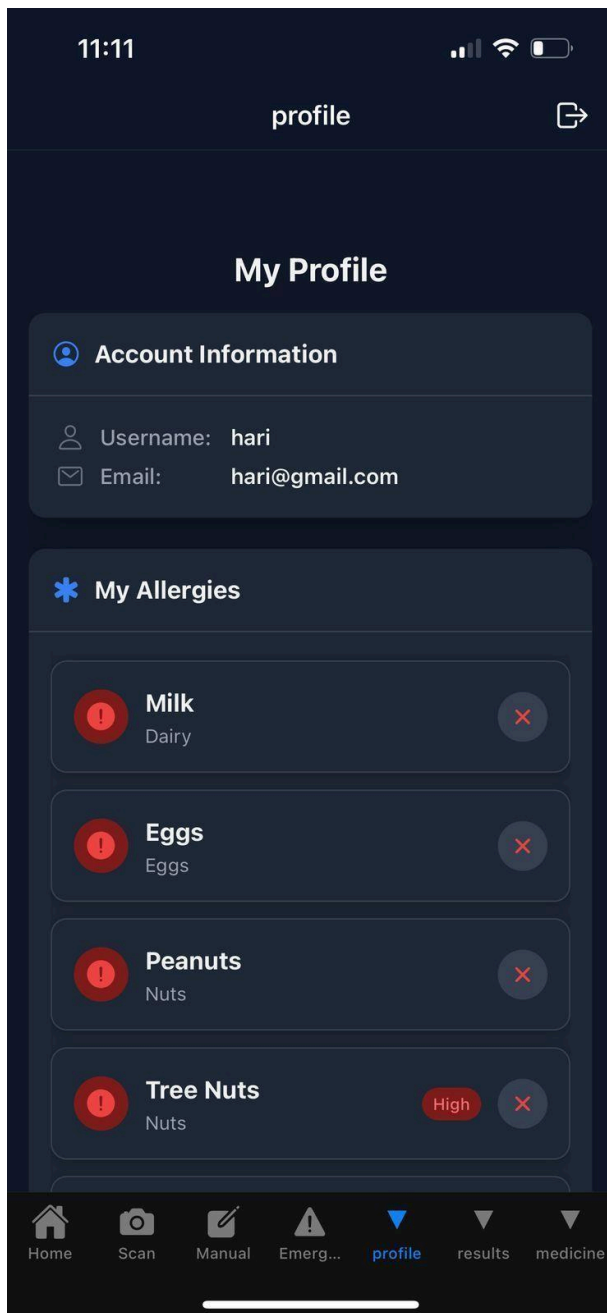


Fig 7.5: Shows the Users profile where the they can customize or change there known allergens and add new ones

7.6 : Medicine List Page

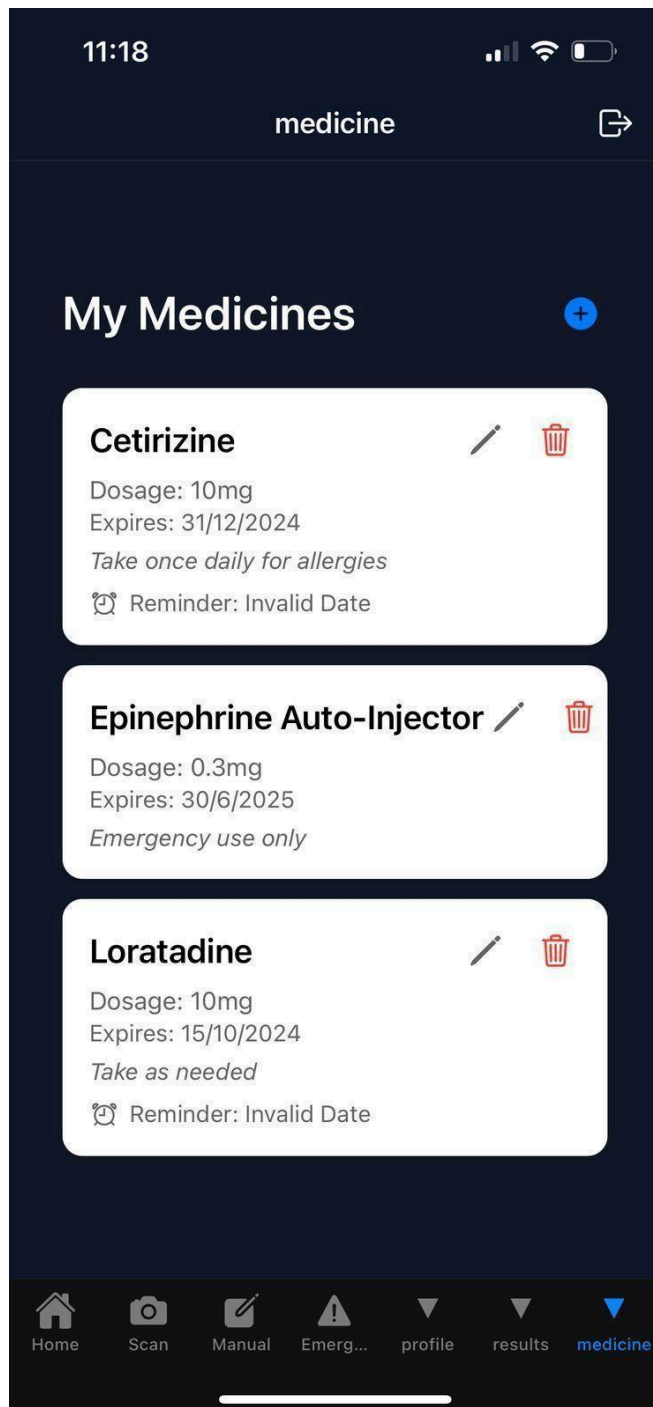


Fig 7.6: Shows the Medicine List page where the user can add any prescribed medicines to the list along with its dosage and expiration dates.

7.7 : Emergency Steps Page

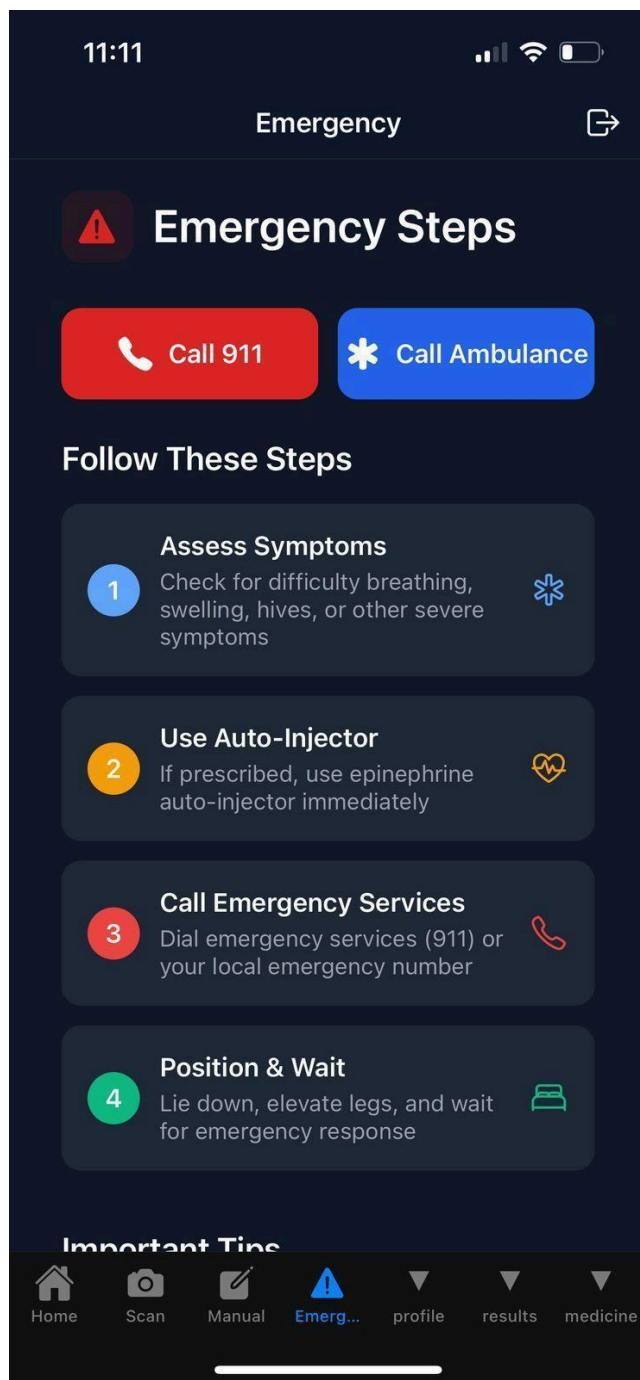


Fig 7.7: Shows all the emergency steps to be taken during an anaphylactic shock situation, the page also includes two buttons to call an Ambulance and Police in case of an emergency.

Figure Descriptions

Fig 7.1: User Registration / Login Page

This screen is the app's entry point. At the top, a stylized syringe icon and the title "Allergy Detection" establish brand identity and purpose. Underneath, the tagline "Scan, detect, and stay safe from allergens" reinforces the core value proposition. Two input fields prompt for "Username or Email" and "Password," each paired with clear icons (user and lock). The primary "Sign In" button is styled in bright blue for prominence. A secondary "Don't have an account? Sign Up" link at the bottom directs new users to the registration flow. Overall, the layout uses high-contrast text on a dark background and generous spacing to ensure readability and accessibility.

Fig 7.2: Home Page

Upon login, the Home Page greets the user by name ("Welcome, hari") and labels itself as "Your allergy management assistant." A prominent card titled "Allergy Detection App" offers a one-line summary and an icon button leading to the scan feature. Below, a "Quick Actions" grid provides four tappable tiles:

1. **Scan Products** (camera icon) – Launches the OCR scanner.
2. **Manual Entry** (pencil icon) – Opens a form for typing ingredients by hand.
3. **Emergency Info** (alert icon) – Displays critical emergency contacts and steps.
4. **Medicine Tracker** (pill icon) – Manages allergy medication schedules.

A bottom navigation bar allows switching instantly between Home, Scan, Manual entry, Emergency, Profile, Results, and Medicine tabs.

Fig 7.3: OCR Scan Product Label Page

This page guides the user through capturing a product label. A live preview window at the top shows the selected label image with a "Capture or select a product label to detect allergens" prompt. Two primary buttons—"Take Photo" (camera) and "Gallery" (image icon)—allow image selection. A third "Scan Label" button initiates OCR. Once scanned, the "Extracted Text" section appears below, displaying the raw ingredient text for user verification before classification.

Fig 7.4: Scan Results Page

After OCR and classification, this page presents detected allergens in two tabs: "Current Scan" and "History." Under "Current Scan," each allergen is shown as a colored pill (e.g., soy, wheat, dairy, egg). User-specific allergens are flagged with "(Your Allergy)" and a warning icon. Below each pill, a confidence score (e.g., 78%) indicates model certainty, followed by the ingredient snippets where the allergen appeared. A highlighted warning box ("This is one of your known allergies. Exercise extreme caution.") provides immediate, personalized risk guidance. The "History" tab (not shown) archives past scan results.

Fig 7.5: User Profile Page

This page lets users manage their personal information and allergy profile. The top "Account

Information” card shows username and email. Beneath, the “My Allergies” list displays each allergy as a card with an icon, name (e.g., Milk, Eggs, Peanuts, Tree Nuts), and severity indicator (e.g., “High”). Each card has a delete (×) button for removal. An “Add Allergy” (+) action (not shown in this screenshot) lets users append new allergens. This centralized view empowers users to keep their profile up to date.

Fig 7.6: Medicine List Page

“My Medicines” presents all user-entered medications in card form. Each card shows:

- **Name:** (e.g., Cetirizine, Epinephrine Auto-Injector, Loratadine)
- **Dosage:** (e.g., 10 mg, 0.3 mg)
- **Expiration Date:** (day/month/year)
- **Notes:** (e.g., “Take once daily for allergies,” “Emergency use only”)
- **Reminder Status:** (shows if a valid reminder is set)

Each card includes edit (pencil) and delete (trash) icons. A floating “+” button enables adding new prescriptions or over-the-counter medications.

Fig 7.7: Emergency Steps Page

This dedicated emergency screen equips users for anaphylactic events. At the top are two large buttons—“Call 911” (red) and “Call Ambulance” (blue)—for instant connection to emergency services. Below, a numbered sequence of critical steps guides the user through:

1. **Assess Symptoms:** Identify breathing difficulties, swelling, hives.
2. **Use Auto-Injector:** Administer epinephrine if prescribed.
3. **Call Emergency Services:** Dial local emergency number.
4. **Position & Wait:** Lie down, elevate legs, and await responders.

Each step uses an icon and concise instruction. Important safety tips or reminders (e.g., “Stay calm,” “Monitor airway”) may follow.

Chapter 7: Conclusion and Future Work

Overall Conclusion

This project, **Allergy Detection via Ingredient Scanning**, aims to address a pressing need for individuals managing food allergies in day-to-day life. By combining **OCR-based label scanning**, **machine learning** for hidden allergen detection, and **user-centric features** (medicine management, emergency guidance, user-specific allergen profiles), we have developed a comprehensive mobile application capable of swiftly identifying potential allergens in product ingredients.

Throughout the implementation:

1. Backend:

- Built using **FastAPI** for a lightweight but robust REST API.
- Incorporated **multi-label classification** (Random Forest with TF-IDF features) to detect complex or scientific allergen terms.
- Utilized straightforward authentication flows (registration, login, token refresh) to personalize each user's allergen experience.

2. Frontend:

- Developed with **React Native** (Expo) to provide a cross-platform mobile solution.
- Implemented an intuitive interface for label scanning (OCR), manual text entry, and medication tracking.
- Ensured a user-friendly experience by integrating quick actions, straightforward flows, and dark mode.

By focusing on real-time text extraction and dynamic allergen analysis, the system significantly reduces the risk of overlooking critical allergen information. It also centralizes essential features—such as emergency steps and medicine reminders—within a single application, eliminating the need for multiple separate apps.

Chapter 8: References

- [1] Smith, J., and Kumar, R. “Food Allergen Detection Using Machine Learning: A Multi-label Classification Approach.” *Journal of Biomedical Informatics*, vol. 108, 2020, p. 103496.
- [2] World Allergy Organization. “Global Allergy Facts and Statistics.” World Allergy Organization, 2021, www.worldallergy.org/.
- [3] Nguyen, T., and Rossi, F. “Improving Food Label Parsing through Domain-Specific Text Processing.” *Proceedings of the NLP for Food Safety Workshop*, 2022.
- [4] Breiman, L. “Random Forests.” *Machine Learning*, vol. 45, no. 1, 2001, pp. 5–32.
- [5] Kwon, J., and Lee, Y. “Hybrid OCR Pipelines for Real-Time Label Interpretation on Mobile Devices.” *Journal of Mobile Computing Research*, vol. 14, no. 4, 2021, pp. 211–223.
- [6] Amrutha, M., and Rani, S. “Smart Food Label Reader Using Tesseract OCR and Image Preprocessing Techniques.” *International Journal of Computer Applications*, vol. 183, no. 7, 2021, pp. 18–24.
- [7] Tanaka, A., and Hayashi, M. “Explainable AI for Food Allergy Warnings.” *Proceedings of the ACM Conference on Health, Inference, and Learning (CHIL)*, 2023.
- [8] Otsu, Nobuyuki. “A Threshold Selection Method from Gray-Level Histograms.” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-9, no. 1, 1979, pp. 62–66.
- [9] Sourvanos, I., et al. “Morphological Approaches to Text Segmentation on Curved Labels.” *Intl. Conference on Document Analysis and Recognition*, 2020.
- [10] Manning, C. D., Raghavan, P., and Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [11] Pedregosa, F., et al. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830.
- [12] Jose, R., and Mathew, R. “Mobile Applications for Allergy Detection: A Comparative Study.” *International Journal of Healthcare Informatics*, vol. 6, no. 2, 2020, pp. 24–30.
- [13] Zhai, S., and Jiang, X. “On-device Machine Learning Inference for Health Apps.” *IEEE Embedded Systems Letters*, vol. 10, no. 3, 2018, pp. 65–68.
- [14] Bradski, G. “The OpenCV Library.” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [15] Tesseract OCR. Accessed from <https://tesseract-ocr.github.io/>.
- [16] Google Developers. “Best Practices for Mobile OCR.” Accessed from <https://developers.google.com/vision/ocr>.

- [17] DeepAI. “TF-IDF Explained.” Accessed from <https://deepai.org/machine-learning-glossary-and-terms/tf-idf>.
- [18] Kaggle. “Allergen Ingredient Dataset.” Accessed from <https://www.kaggle.com>.
- [19] FastAPI Documentation. Accessed from <https://fastapi.tiangolo.com>.
- [20] React Native Documentation. Accessed from <https://reactnative.de>